ADR-0007 : Uploads résumables par fragments (chunked upload)

- **Statut :** Accepté
- **Date :** 2026-07-21
- **Décideurs :** Driss OUAAZIZ — Software Architect

Contexte et problème

FileFlow doit accepter le dépôt de **fichiers volumineux** sur des réseaux potentiellement instables. Un upload en une seule requête multipart présente deux difficultés :

- une **coupure réseau** en cours de transfert impose de tout recommencer ;
- une requête unique de très grande taille peut mettre en tension la mémoire du serveur si elle n'est pas maîtrisée.

Il faut donc un mécanisme permettant de **reprendre** un transfert interrompu là où il s'est arrêté, tout en **bornant la mémoire** consommée côté serveur et en préservant les contrôles d'accès et de quota.

Facteurs de décision

- Reprise après coupure sans retransmettre les octets déjà reçus.
- Empreinte mémoire serveur bornée, indépendante de la taille du fichier.
- Robustesse aux sessions abandonnées (pas de fichiers temporaires orphelins).
- Maintien des contrôles d'autorisation et de quota au moment opportun.

Options envisagées

1. **Upload multipart en une requête** — Simple, mais aucune reprise possible et maîtrise mémoire délicate sur les gros fichiers. Conservé comme voie simple pour les petits fichiers, insuffisant pour les volumineux.
2. **Découpage en fragments avec état côté serveur** *(retenu)* — Le client découpe le fichier et transmet des fragments ordonnés ; le serveur assemble sur disque et sait indiquer l'offset courant pour reprendre.

Décision

Implémenter un **protocole d'upload résumable par fragments** exposant les opérations `init` / `chunk` / `status` / `finish` / `abort` :

- `init` valide en amont l'accès en écriture, la liste noire d'extensions et le quota, crée le fichier temporaire `.part` et renvoie un identifiant de session.
- `chunk` ajoute un fragment **à l'offset attendu** ; un décalage renvoie un conflit (409) accompagné de l'offset correct, permettant au client de se resynchroniser. Le corps de la requête de fragment est plafonné à `MAX_CHUNK_BYTES = 32 Mio` (limite de corps par route) afin d'empêcher qu'un fragment démesuré n'épuise la mémoire. Les écritures concurrentes sur une même session sont sérialisées par un verrou.

- `status` renvoie le nombre d'octets déjà reçus, base de la reprise.
- `finish` vérifie la complétude, **re-valide l'accès en écriture** (les permissions ont pu changer depuis `init`), archive une éventuelle version existante puis déplace le `.part` en place et met à jour quota et audit.
- `abort` supprime la session et son fichier temporaire.

Gestion du cycle de vie :

- Les **sessions vivent en mémoire uniquement** ; seuls les octets résident sur disque dans le `.part`.
- Un **reaper périodique** élimine les sessions inactives au-delà de `SESSION_TTL_SECS = 6 heures` et supprime leur fichier temporaire.
- Au **démarrage**, les fichiers `.part` orphelins (issus d'un arrêt antérieur) sont nettoyés, puisque les sessions ne survivent pas au redémarrage.

Conséquences

Positives

- Reprise d'un transfert interrompu sans retransmission des octets reçus.
- Empreinte mémoire serveur bornée (fragments plafonnés, données sur disque).
- Absence de fichiers temporaires orphelins (reaper + nettoyage au démarrage).
- Contrôles d'accès et de quota appliqués au bon moment, y compris re-validation à la finalisation.

Négatives

- Protocole plus complexe côté client (découpage, gestion des offsets et des reprises) que l'upload en une requête.
- L'état de session en mémoire est **non partagé** : un déploiement multi-instance imposerait une affinité de session ou un état externalisé.

Risques résiduels

- Un redémarrage serveur pendant un upload invalide la session en cours ; le client doit relancer depuis un nouvel `init` (les `.part` orphelins étant nettoyés au démarrage).

Références

- Code : `backend/src/uploads.rs`, routes `/api/uploads/*` dans `backend/src/main.rs`, client `frontend/src/api.ts` (`uploadResumable`).
- ADR-0001 (persistance) — l'état de session est délibérément conservé hors du magasin SQLite.